The Journal of Open Source Software

Pydisort: A Modern Python Interface for Discrete Ordinates Radiative Transfer (DISORT)

Zoey Hu [1] **and Cheng Li** [1]

1 University of Michigan, Ann Arbor, MI, USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 02 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Radiative transfer in layered media is central to atmospheric and planetary science. DISORT (Discrete Ordinate Radiative Transfer) is a widely-used algorithm that calculates the scattering and absorption of radiation in a plane-parallel medium. The original DISORT algorithm was implemented in `Fortran` (Stamnes et al., 1988) with static memory allocation. The dimension of the internal arrays, such as the number of atmospheric layers and the number of radiation streams must be specified at compile time. Despite of this limitation, the original `Fortran` program has been widely used in Earth and planetary atmospheres (e.g. (Clough et al., 2005; Komacek et al., 2022; Lee, 2024; Li et al., 2018; Tan & Showman, 2021; Zhang et al., 2015)).

With the increasing adoption of `Python` in scientific computing, over the years, many efforts have been made to wrap the original `Fortran` code in `Python` to provide a more user-friendly interface. Notable examples include (1) chanGimeno's, (2) SeregaOsipov's, (3) danielkoll's (Koll & Cronin, 2019), and (4) mjwolf's. All of them are named `pydisort` and are based on the original `Fortran` code. Their `Python` interfaces are enabled by `f2py` (Van Der Walt et al., 2011) functionality, which is a part of the `NumPy` package and provides a convenient way to call `Fortran` subroutines from `Python`. A specific implementation of `pydisort` for near-IR radiative transfer in Titan's atmosphere is provided through the atmosphere package by Ádámkovics et al. (2016).

Independently, Ho (2024) developed a pure `Python` implementation of the DISORT algorithm, which is less efficient than the `Fortran` implementation but is easier to use for pedagogical and exploratory purposes.

Here, we provide a `pip`-installable modern `Python` interface to the original DISORT implementation. Instead of starting from the original `Fortran` code, we use the `C` implementation of DISORT by Buras et al. (2011) as the computing backend. The `cdisort` library improves upon the original `Fortran` code by using dynamic memory allocation, allowing for a more flexible implementation and easier adaptation to `Python`. `cdisort` has been an integral component of the `libRadtran` radiative transfer software package (Emde et al., 2016), but is lesser known in the atmospheric science community than the original `Fortran` implementation because the previous generation of atmospheric models were mostly written in `Fortran`.

Our `Pydisort` package differs from the previous `pydisort` packages in the following ways:

1. We use the `C` implementation of DISORT as the backend, which allows for dynamic memory allocation during runtime.
2. We use `PyTorch` (Paszke, 2019)'s `tensor` data structure for memory management, paving the way for heterogeneous computing and automatic parallelization.
3. We create an intermediate `C++` layer to the backend `C`-library. The intermediate layer handles pre- and post-processing of the raw `cdisort` data structures.
4. We leverage `Pybind11` (Jakob, 2024) to interface between `Python` API calls and our intermediate `C++` interface. `Pybind11` is a header-only modern alternative to `f2py` with

graceful type-handling, casting and memory management.

5. We design software architecture to support building the C/C++ backend libraries as shared libraries, linking them to `libtorch` and `Python`, and distributing them as `pip`-installable packages for various platforms, including `Linux` and `MacOS`.
6. We automate the building and distribution process using `cibuildwheel` on Mac images and Linux images with `glibc 2.28+`, which is the minimum version of `glibc` required by `PyTorch v2.7+`.
7. We dynamically determine the `CXX11_ABI` version from the upstream `libtorch` library. Currently with `libtorch v2.7`, the `CXX11_ABI` version is `1` for Linux distributions and `0` for MacOS distributions.
8. We provide two frontends: (1) a `Python` interface and (2) a `C++` interface. The former is useful for users who want to use the package in `Python` and take advantage of the machine learning capabilities enabled by `PyTorch`, while the latter is useful for users who want to integrate the package into their own `C/C++` packages.
9. We automate the Continuous Integration (CI) and Continuous Distribution (CD) processes using `GitHub Actions` to ensure that minimal human effort is required for maintaining the package.
10. We adhere to the `PEP 8` (Van Rossum et al., 2001) style guide for `Python` code, making the program a `Python-first` experience. The function calls make frequent use of `Python` features such as keyword arguments and named arguments, which are idiomatic to `Python` users.

Statement of need

Tools for calculating atmospheric radiative transfer are essential for a wide range of applications in atmospheric modeling and remote sensing. As demonstrated in the Summary section, the original `Fortran` implementation of DISORT has been widely used in research. To accommodate the growing popularity of `Python` in the scientific community, many groups have developed their own Python wrappers for the original Fortran implementation of DISORT. However, few have reached the same level of maturity, efficiency, and usability as our package.

Our package manages to distribute `pip`-installable pre-built library that wraps the DISORT algorithm implemented in a lower-level compiled language such as `C` or `Fortran`. Our package

1. saves users from the hassle of setting up the build environment and the tool chains.
2. skips the compilation process because we ship the compiled shared libraries with the package on PyPI.
3. achieves the same level of performance as the original `C/Fortran` implementation.
4. provides a modern `Python` interface that is easy to use and understand.

The previous effort of educational purpose has been made by Ho (2024), which is a pure `Python` implementation of the DISORT algorithm and Ricchiazzi et al. (1998), which is a `Fortran77` implementation. Since our backend subroutines are implemented in `C`, our code runs as fast as the original `Fortran` implementation, while the `Python` interface lowers the barrier to entry for users who are not familiar with `Fortran` or `C`.

Additionally, `Pydisort` improves upon the `C/Fortran` implementations by enabling parallelization over wavelengths and columns. For plane-parallel atmospheres, the radiative transfer equation is separable by columns and by wavelengths/wavenumbers. However, looping over columns and wavelengths in `Python` will significantly slow down the code. Therefore, we leverage the `PyTorch` tensor data structure and its `TensorIterator` functionality to enable concurrent execution over columns and wavelengths. Future improvements of the package will seamlessly allow GPU acceleration of the radiative transfer calculation when the original `C` backend is adapted to execute on GPUs.

Acknowledgements

We acknowledge Dr. Timothy E. Dowling for his work on migrating the original FORTRAN version of DISORT to C, which is the basis for our implementation. We acknowledge Dr. Xi Zhang and Dr. Tianhao Le for initiating the project and testing the code.

References

Ádámkovics, M., Mitchell, J. L., Hayes, A. G., Rojo, P. M., Corlies, P., Barnes, J. W., Ivanov, V. D., Brown, R. H., Baines, K. H., Buratti, B. J., & others. (2016). Meridional variation in tropospheric methane on titan observed with AO spectroscopy at keck and VLT. *Icarus*, *270*, 376–388.

Buras, R., Dowling, T., & Emde, C. (2011). New secondary-scattering correction in DISORT with increased efficiency for forward scattering. *Journal of Quantitative Spectroscopy and Radiative Transfer*, *112*(12), 2028–2034.

Clough, S. A., Shephard, M. W., Mlawer, E. J., Delamere, J., Iacono, M. J., Cady-Pereira, K., Boukabara, S., & Brown, P. D. (2005). Atmospheric radiative transfer modeling: A summary of the AER codes. *Journal of Quantitative Spectroscopy and Radiative Transfer*, *91*(2), 233–244.

Emde, C., Buras-Schnell, R., Kylling, A., Mayer, B., Gasteiger, J., Hamann, U., Kylling, J., Richter, B., Pause, C., Dowling, T., & others. (2016). The libRadtran software package for radiative transfer calculations (version 2.0. 1). *Geoscientific Model Development*, *9*(5), 1647–1672.

Ho, D. J. (2024). PythonicDISORT: A python reimplementation of the discrete ordinate radiative transfer package DISORT. *Journal of Open Source Software*, *9*(103), 6442.

Jakob, W. (2024). *pybind11 documentation*.

Koll, D. D., & Cronin, T. W. (2019). Hot hydrogen climates near the inner edge of the habitable zone. *The Astrophysical Journal*, *881*(2), 120.

Komacek, T. D., Tan, X., Gao, P., & Lee, E. K. (2022). Patchy nightside clouds on ultra-hot jupiters: General circulation model simulations with radiatively active cloud tracers. *The Astrophysical Journal*, *934*(1), 79.

Lee, E. K. (2024). Testing approximate infrared scattering radiative-transfer methods for hot jupiter atmospheres. *The Astrophysical Journal*, *965*(2), 115.

Li, C., Le, T., Zhang, X., & Yung, Y. L. (2018). A high-performance atmospheric radiation package: With applications to the radiative energy budgets of giant planets. *Journal of Quantitative Spectroscopy and Radiative Transfer*, *217*, 353–362.

Paszke, A. (2019). Pytorch: An imperative style, high-performance deep learning library. *arXiv Preprint arXiv:1912.01703*.

Ricchiazzi, P., Yang, S., Gautier, C., & Sowle, D. (1998). SBDART: A research and teaching software tool for plane-parallel radiative transfer in the earth's atmosphere. *Bulletin of the American Meteorological Society*, *79*(10), 2101–2114.

Stamnes, K., Tsay, S.-C., Wiscombe, W., & Jayaweera, K. (1988). Numerically stable algorithm for discrete-ordinate-method radiative transfer in multiple scattering and emitting layered media. *Applied Optics*, *27*(12), 2502–2509.

Tan, X., & Showman, A. P. (2021). Atmospheric circulation of brown dwarfs and directly imaged exoplanets driven by cloud radiative feedback: Global and equatorial dynamics. *Monthly Notices of the Royal Astronomical Society*, *502*(2), 2198–2219.

DRAFT

Van Der Walt, S., Colbert, S. C., & Varoquaux, G. (2011). The NumPy array: A structure for efficient numerical computation. *Computing in Science & Engineering*, *13*(2), 22–30.

Van Rossum, G., Warsaw, B., & Coghlan, N. (2001). PEP 8–style guide for python code. *Python. Org*, *1565*, 28.

Zhang, X., West, R. A., Irwin, P. G., Nixon, C. A., & Yung, Y. L. (2015). Aerosol influence on energy balance of the middle atmosphere of jupiter. *Nature Communications*, *6*(1), 10231.

DRAFT